

URL Shortener System Design

Table of Contents

- 1. [Overview](#)
 - 2. [Requirements](#)
 - 3. [Capacity Estimation](#)
 - 4. [Schema Design](#)
 - 5. [ASCII ER Diagram](#)
 - 6. [Indexing Strategy](#)
 - 7. [Query Patterns](#)
 - 8. [Scaling Strategy](#)
 - 9. [Tradeoffs](#)
 - 10. [Interview Discussion Points](#)
 - 11. [Common Interview Follow-ups](#)
 - 12. [Performance Considerations](#)
 - 13. [Cross-References](#)
-

Overview

A URL shortener (like bit.ly or tinyurl.com) maps a short alphanumeric code to a long URL. The system has an extreme read-to-write ratio (100:1 or more) since each short URL may be clicked thousands of times after being created once. The critical path — resolving a short code to a long URL — must be as fast as possible, typically served from cache with PostgreSQL as the authoritative source of truth.

Requirements

Functional Requirements

- Create a short URL from a long URL
- Redirect from short URL to long URL
- Custom aliases (user-chosen short codes)
- Expiry: short URLs can expire after a date or a number of clicks
- Analytics: track clicks, referrers, geolocation, device type
- User accounts: manage your own short URLs, view analytics
- API access for programmatic usage

Non-Functional Requirements

- 100M+ active short URLs
 - 100:1 read:write ratio
 - P99 redirect latency < 10ms (served from cache)
 - P99 redirect latency < 50ms (cache miss, hit DB)
 - 99.99% uptime for redirects
 - Short codes: globally unique, hard to guess, 6–8 characters
 - Analytics updates can be eventually consistent (seconds lag acceptable)
-

Capacity Estimation

Metric	Estimate
--------	----------

Short URLs created per day	1M
Redirects per day	100M
Peak redirects per second	2,000
Short URL size	6 characters (base62: a-z, A-Z, 0-9)
Possible combinations (base62^6)	56.8 billion
Active short URLs	100M
Click events per day	100M

Storage Estimates

Table	Rows	Row Size	Total
short_urls	365M (lifetime)	600B	219GB
click_events	36.5B (1yr)	200B	7.3TB/yr
click_hourly_stats	365M (1yr)	150B	55GB/yr
users	10M	500B	5GB

Bandwidth: 100M redirects × 500B/request = 50GB/day inbound + outbound (minimal, redirects are 301/302 with no body).

Cache requirement: 100M active URLs × 600B = 60GB — fits in a Redis cluster comfortably.

Observation: The `click_events` table is the storage bottleneck. At 100M/day × 200B = 20GB/day. Separate this from the core URL table.

Schema Design

```
-- =====
-- EXTENSIONS
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- =====
-- ENUM TYPES
-- =====

CREATE TYPE url_status AS ENUM ('active', 'expired', 'disabled', 'deleted');
CREATE TYPE click_device AS ENUM ('desktop', 'mobile', 'tablet', 'bot', 'unknown');

-- =====
-- USERS
-- =====

CREATE TABLE users (
    user_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

    email            VARCHAR(320)    NOT NULL,
    password_hash    VARCHAR(255)    NOT NULL,
    api_key          VARCHAR(64)      NOT NULL,
    plan             VARCHAR(20)      NOT NULL DEFAULT 'free',    --
'free','pro','business'
    url_count        INT              NOT NULL DEFAULT 0,
    monthly_clicks   BIGINT           NOT NULL DEFAULT 0,
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_users_email UNIQUE (email),
    CONSTRAINT uq_users_api_key UNIQUE (api_key)
);

-- =====
-- SHORT URLS (Core Table)
-- =====
CREATE TABLE short_urls (
    url_id           UUID              PRIMARY KEY DEFAULT uuid_generate_v4(),
    short_code       VARCHAR(20)       NOT NULL,
    long_url         TEXT              NOT NULL,
    user_id          UUID              REFERENCES users(user_id) ON DELETE SET NULL,
    status           url_status        NOT NULL DEFAULT 'active',
    title            VARCHAR(500),     -- auto-fetched from the target page
    -- Expiry options
    expires_at       TIMESTAMPTZ,
    max_clicks       BIGINT,           -- expire after N clicks, NULL = unlimited
    -- Stats (denormalized for fast reads)
    total_clicks     BIGINT            NOT NULL DEFAULT 0,
    unique_clicks    BIGINT            NOT NULL DEFAULT 0,
    last_clicked_at  TIMESTAMPTZ,
    -- Custom domain support
    domain           VARCHAR(255)      NOT NULL DEFAULT 'short.ly',
    -- Metadata
    tags             TEXT[]            DEFAULT '{}',
    notes            TEXT,
    created_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_short_code UNIQUE (domain, short_code),
    CONSTRAINT chk_long_url_length CHECK (length(long_url) <= 8192),
    CONSTRAINT chk_short_code_chars CHECK (short_code ~ '^[a-zA-Z0-9_-]+$')
);

-- =====
-- SHORT CODE GENERATION
-- =====
-- Approach 1: Counter-based (Zookeeper / PostgreSQL sequence)
CREATE SEQUENCE url_code_seq START 1 INCREMENT 1 CACHE 1000;

CREATE OR REPLACE FUNCTION int_to_base62(n BIGINT) RETURNS VARCHAR AS $$
DECLARE
    chars TEXT := 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
    result TEXT := '';

```

```

        rem INT;
BEGIN
    IF n = 0 THEN RETURN 'a'; END IF;
    WHILE n > 0 LOOP
        rem := n % 62;
        result := substr(chars, rem + 1, 1) || result;
        n := n / 62;
    END LOOP;
    RETURN result;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

-- Auto-generate short_code if not provided
CREATE OR REPLACE FUNCTION generate_short_code()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.short_code IS NULL OR NEW.short_code = '' THEN
        NEW.short_code := int_to_base62(nextval('url_code_seq'));
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_url_short_code
BEFORE INSERT ON short_urls
FOR EACH ROW EXECUTE FUNCTION generate_short_code();

-- =====
-- CLICK EVENTS (Write-Heavy, Time-Series)
-- =====

CREATE TABLE click_events (
    event_id          UUID          NOT NULL DEFAULT uuid_generate_v4(),
    short_code        VARCHAR(20)   NOT NULL,
    url_id            UUID          NOT NULL REFERENCES short_urls(url_id),
    clicked_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
    -- Request metadata
    ip_address        INET,
    user_agent        TEXT,
    referer           TEXT,
    -- Parsed/enriched (done async by background job)
    device_type        click_device NOT NULL DEFAULT 'unknown',
    browser            VARCHAR(100),
    os                 VARCHAR(100),
    country_code       CHAR(2),
    city               VARCHAR(100),
    is_unique          BOOLEAN       NOT NULL DEFAULT FALSE, -- unique per visitor
    per day
    PRIMARY KEY (event_id, clicked_at)
) PARTITION BY RANGE (clicked_at);

CREATE TABLE click_events_2024_01 PARTITION OF click_events
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

```

```

-- Automate with pg_partman

-- =====
-- AGGREGATED STATISTICS (Pre-computed)
-- =====

CREATE TABLE click_hourly_stats (
    url_id          UUID          NOT NULL REFERENCES short_urls(url_id),
    hour_start      TIMESTAMPTZ   NOT NULL,
    total_clicks    INT           NOT NULL DEFAULT 0,
    unique_clicks   INT           NOT NULL DEFAULT 0,
    PRIMARY KEY (url_id, hour_start)
) PARTITION BY RANGE (hour_start);

CREATE TABLE click_daily_stats (
    url_id          UUID          NOT NULL REFERENCES short_urls(url_id),
    stat_date       DATE          NOT NULL,
    total_clicks    INT           NOT NULL DEFAULT 0,
    unique_clicks   INT           NOT NULL DEFAULT 0,
    top_countries   JSONB,        -- {"US":1200,"GB":400,...}
    top_devices     JSONB,
    top_referers    JSONB,
    PRIMARY KEY (url_id, stat_date)
) PARTITION BY RANGE (stat_date);

CREATE TABLE click_country_stats (
    url_id          UUID          NOT NULL REFERENCES short_urls(url_id),
    stat_date       DATE          NOT NULL,
    country_code    CHAR(2)       NOT NULL,
    clicks          INT           NOT NULL DEFAULT 0,
    PRIMARY KEY (url_id, stat_date, country_code)
);

-- =====
-- CUSTOM DOMAINS
-- =====

CREATE TABLE custom_domains (
    domain_id       UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id         UUID          NOT NULL REFERENCES users(user_id),
    domain          VARCHAR(255)  NOT NULL,
    verified        BOOLEAN       NOT NULL DEFAULT FALSE,
    verification_token VARCHAR(100),
    verified_at     TIMESTAMPTZ,
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_custom_domain UNIQUE (domain)
);

-- =====
-- URL BLACKLIST (Spam / Malware)
-- =====

CREATE TABLE url_blacklist (
    entry_id        UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    pattern         TEXT          NOT NULL,

```

```

    pattern_type    VARCHAR(20)    NOT NULL DEFAULT 'exact', --
    'exact', 'prefix', 'regex'
    reason          VARCHAR(200),
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT uq_blacklist_pattern UNIQUE (pattern, pattern_type)
);

```

ASCII ER Diagram

```

+-----+      +-----+      +-----+
| users |1-----N| short_urls |1-----N|click_evts |
+-----+      +-----+      +-----+
                        |1
                        |N
                        +-----+      +-----+
                        |click_hourly |      |click_daily_stats |
                        |stats         |      +-----+
                        +-----+
                        |
                        +-----+
                        |click_country |
                        |stats         |
                        +-----+

+-----+      +-----+
| users |1-----N| custom_domains |
+-----+      +-----+

```

Indexing Strategy

```

-- Short URL: critical redirect path
CREATE INDEX idx_short_urls_code    ON short_urls (domain, short_code);
-- This is the single most important index – every redirect hits it

-- Short URL: user's URL list
CREATE INDEX idx_short_urls_user    ON short_urls (user_id, created_at DESC)
    WHERE user_id IS NOT NULL;

-- Short URL: expiry job
CREATE INDEX idx_short_urls_expiry  ON short_urls (expires_at)
    WHERE status = 'active' AND expires_at IS NOT NULL;

-- Short URL: click limit enforcement
CREATE INDEX idx_short_urls_clicks  ON short_urls (total_clicks, max_clicks)
    WHERE max_clicks IS NOT NULL AND status = 'active';

-- Click events: analytics queries
CREATE INDEX idx_click_events_url   ON click_events (url_id, clicked_at DESC);

```

```

CREATE INDEX idx_click_events_code    ON click_events (short_code, clicked_at DESC);
CREATE INDEX idx_click_events_geo     ON click_events (country_code, clicked_at DESC)
    WHERE country_code IS NOT NULL;

-- Daily stats: URL dashboard
CREATE INDEX idx_daily_stats_url      ON click_daily_stats (url_id, stat_date DESC);

-- Users: API authentication
CREATE INDEX idx_users_api_key        ON users (api_key);

-- Blacklist: fast lookup on create
CREATE INDEX idx_blacklist_pattern    ON url_blacklist (pattern_type, pattern);

```

Query Patterns

Q1 — Redirect Lookup (most critical, sub-millisecond target)

```

-- In production: served from Redis cache. This is the fallback.
SELECT
    url_id, long_url, status, expires_at, max_clicks, total_clicks
FROM short_urls
WHERE domain = $domain
    AND short_code = $code
    AND status = 'active';
-- After fetching: check expires_at and max_clicks in application

```

Q2 — Create Short URL

```

-- Check blacklist first (application logic)
-- Then insert:
INSERT INTO short_urls (long_url, user_id, short_code, expires_at, max_clicks, tags)
VALUES ($long_url, $user_id, COALESCE($custom_code, NULL), $expires_at, $max_clicks,
    $tags)
RETURNING url_id, short_code, domain;

```

Q3 — Record Click (async, batched in production)

```

-- Single click insert (production uses bulk inserts from a queue)
INSERT INTO click_events (short_code, url_id, ip_address, user_agent, referer,
    device_type, country_code)
VALUES ($code, $url_id, $ip, $ua, $referer, $device, $country)
ON CONFLICT DO NOTHING; -- idempotent on event_id

-- Async batch update of counter
UPDATE short_urls
SET total_clicks    = total_clicks + $batch_count,
    last_clicked_at = $last_time,

```

```
        updated_at      = now()
WHERE url_id = $url_id;
```

Q4 — User's URL Dashboard

```
SELECT
    s.url_id, s.short_code, s.domain, s.long_url,
    s.title, s.status, s.total_clicks,
    s.last_clicked_at, s.created_at,
    s.expires_at, s.tags
FROM short_urls s
WHERE s.user_id = $user_id
ORDER BY s.created_at DESC
LIMIT 50 OFFSET $offset;
```

Q5 — URL Analytics: Hourly Breakdown

```
SELECT
    hour_start,
    total_clicks,
    unique_clicks
FROM click_hourly_stats
WHERE url_id = $url_id
    AND hour_start >= $from_date
    AND hour_start < $to_date
ORDER BY hour_start ASC;
```

Q6 — URL Analytics: Country Breakdown

```
SELECT
    country_code,
    SUM(clicks) AS total
FROM click_country_stats
WHERE url_id = $url_id
    AND stat_date BETWEEN $from AND $to
GROUP BY country_code
ORDER BY total DESC
LIMIT 20;
```

Q7 — Hourly Stats Rollup Job

```
INSERT INTO click_hourly_stats (url_id, hour_start, total_clicks, unique_clicks)
SELECT
    url_id,
    DATE_TRUNC('hour', clicked_at) AS hour_start,
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE is_unique) AS unique_c
```



```
FROM click_events
WHERE clicked_at >= $hour_start
    AND clicked_at < $hour_end
GROUP BY url_id, DATE_TRUNC('hour', clicked_at)
ON CONFLICT (url_id, hour_start)
DO UPDATE SET
    total_clicks = EXCLUDED.total_clicks,
    unique_clicks = EXCLUDED.unique_clicks;
```

Q8 — Expire Overdue URLs (scheduled job)

```
UPDATE short_urls
SET status = 'expired', updated_at = now()
WHERE status = 'active'
    AND expires_at < now()
RETURNING url_id, short_code;
```

Q9 — Check Custom Code Availability

```
SELECT EXISTS (
    SELECT 1 FROM short_urls
    WHERE domain = $domain AND short_code = $code
) AS taken;
```

Q10 — Top URLs by Clicks (admin dashboard)

```
SELECT
    s.short_code, s.domain, s.long_url, s.total_clicks,
    s.created_at,
    u.email AS owner_email
FROM short_urls s
LEFT JOIN users u ON u.user_id = s.user_id
WHERE s.created_at >= now() - INTERVAL '7 days'
ORDER BY s.total_clicks DESC
LIMIT 100;
```

Scaling Strategy

The Redirect Path is Everything

Every technique below aims to make the redirect (`short_code` → `long_url`) faster and more scalable.

Layer 1: Redis Cache (essential from day 1)

```
Client Request → Load Balancer → API Server
API Server → Redis GET short_code → long_url (hit: sub-millisecond)
```

→ Miss: query PostgreSQL, SET in Redis (TTL 24h)
→ 301 Redirect

Cache key: `url:{domain}:{short_code}` → JSON of `{long_url, status, expires_at, max_clicks}`

Hit rate target: 99%+ (most popular URLs cached; long tail handled by DB) **Redis eviction:** LRU, max 60GB memory

Layer 2: Read Replicas

At 2,000 redirects/second, Redis should absorb ~99%. The 20 cache misses/second can be handled by a single PostgreSQL primary. Add read replicas when `pg_stat_activity` shows contention.

Layer 3: Click Event Buffering

At 100M clicks/day = 1,160 writes/second. Never write to PostgreSQL synchronously in the redirect path:

Click Event → Kafka topic 'url.clicks'
→ Consumer 1: batch inserts to `click_events` (every 30s)
→ Consumer 2: increments Redis counter for real-time stats
→ Consumer 3: updates `click_hourly_stats`

Layer 4: Sharding (at 1B+ URLs)

Shard `short_urls` by `hash(short_code) % N`. Each shard handles a slice of the keyspace. The routing service maps `short_code` to shard based on a consistent hash ring.

Short Code Generation at Scale

Single PostgreSQL sequence becomes a bottleneck at very high write rates. Alternatives:

1. **Pre-generated pool:** Generate 1M short codes in a `available_codes` table; workers claim them atomically with `DELETE ... RETURNING SKIP LOCKED`.
2. **Distributed counter:** Zookeeper or etcd allocates ranges (e.g., worker 1 gets range 1–1000000, worker 2 gets 1000001–2000000).
3. **Hash-based:** MD5(url + salt), take first 6 chars, retry on collision.

Tradeoffs

Decision	Chosen	Alternative	Why
Counter-based short codes	PostgreSQL sequence	Random hash	Guaranteed uniqueness, no collision; shorter codes; sequential IDs are predictable (tradeoff: enumerable)
301 vs. 302 redirect	302 (temporary)	301 (permanent)	301 is cached by browsers forever — you lose analytics. 302 ensures every click goes through the server
Async click recording	Kafka → batch write	Synchronous INSERT	Synchronous write would double redirect latency; async decouples analytics from availability

Denormalized total_clicks	Stored counter	Query click_events	COUNT(*) on billions of rows per URL fetch is unacceptable; update counter in batch
Daily+hourly stats tables	Pre- aggregated	On-demand GROUP BY	Analytics page would scan billions of click_events; pre-aggregation reduces query time to milliseconds

Interview Discussion Points

- 1. Why use a database at all for redirects?** PostgreSQL is not in the hot path — Redis is. PostgreSQL is the source of truth for URL creation, analytics, and recovery after cache failure. The database tier must handle cache warmup after a Redis flush, analytics aggregation, and URL management.
- 2. How do you guarantee short code uniqueness across distributed servers?** Option A: PostgreSQL sequence (centralized, atomic). Option B: Pre-allocated ranges distributed to servers. Option C: UUID truncated to 8 chars with collision retry. For correctness, PostgreSQL sequence is simplest; for extreme write scale, distributed range allocation.
- 3. How do you handle 301 vs. 302 redirects?** Use 302 for analytics accuracy — browsers cache 301s permanently and never follow them again through your server. Use 301 for maximum performance (after sufficient data collection) when analytics aren't needed.
- 4. How does the analytics pipeline work?** Redirects push click events to Kafka. A stream processor enriches them (geo-IP lookup, user-agent parsing), then writes batches to `click_events`. Separate aggregation jobs roll up hourly and daily stats. The redirect itself only touches Redis — zero DB writes in the hot path.

Common Interview Follow-ups

Q: How would you handle a viral short URL with 1M clicks per second? A: Redis can handle millions of GET operations per second per node. The main consideration is cache key hot spotting — a single URL being hit by a Redis cluster may route all requests to one shard. Use client-side read replication or hash-tag-based Redis clustering to distribute the load across multiple Redis nodes.

Q: How do you prevent abuse (spam, phishing)? A: Check `url_blacklist` on creation (exact match, prefix match, regex). Integrate Google Safe Browsing API for real-time malware/phishing detection. Rate limit creation per API key. For anonymous users, CAPTCHA after N URLs per IP per day.

Q: How would you implement link previews (title, description, thumbnail)? A: On URL creation, enqueue a background job that fetches the target URL, parses `<title>` and `<meta og:*>` tags, and updates `short_urls.title`. Store the thumbnail URL. This is done asynchronously so it never blocks URL creation.

Performance Considerations

- Redis pipeline:** Batch Redis commands in the redirect handler — one pipeline call for GET + increment counter vs. two separate round trips.
- Partition pruning on click_events:** Always include `clicked_at >= $date` in analytics queries. Without this, all monthly partitions are scanned.
- B-Tree index cardinality:** The `(domain, short_code)` index has near-100% cardinality — every redirect is an exact lookup. This is ideal for B-Tree: $O(\log N)$ lookup on 100M rows = ~30 index page

reads, ~microseconds.

- **Connection pooling for burst:** Viral URLs cause burst traffic. PgBouncer absorbs connection spikes; pool size of 50 connections can serve thousands of concurrent requests.
 - **Click event INSERT throughput:** At 1,160/second baseline, use COPY or multi-row INSERT from the Kafka consumer batch. COPY is 5–10x faster than individual INSERTs.
-

Cross-References

- See `18_Architecture_Case_Studies/05_saas_product.md` for API rate limiting patterns
- See `07_analytics_platform_design.md` for click event analytics pipeline
- See `08_system_design_framework.md` for interview presentation framework
- PostgreSQL sequences: Chapter 3 of this guide
- Partitioning: Chapter 5 of this guide
- Caching strategies: Chapter 12 of this guide